Université Gustave Eiffel

Laboratoire d'Informatique Gaspard Monge

Space Invader

Computer Vision

Auteurs : Badredin BELACHHEB & Janelle TOURNEUR & Lisa PIN

Encadrant : Florian VALADE

Champs-sur-Marne — Année universitaire 2025–2026

Table des matières

Computer Vision : Space Invaders **1**

1 Collecte des données 1

2 Bases 1

2.1 Neurone artificiel 1

2.2 Perceptron 2

Computer Vision : Space Invaders

1 Collecte des données

Le script `collect_dataset.py` permet de créer notre dataset de photos qui servira à l'entraînement du modèle. Il active la webcam et enregistre l'image entière en la classant dans des dossiers selon le geste effectué et selon un split *train/validation*. L'utilisateur contrôle la collecte au clavier :

- Appuyer sur une touche entre `0` et `5` pour sélectionner une classe (`NONE`, `LEFT`, `RIGHT`, `FIRE`, `ENTER`, `QUIT`)
- Appuyer sur `c` pour activer ou désactiver la capture (5 images par seconde)
- Appuyer sur `v` pour basculer entre le mode *train* et le mode *validation*
- Appuyer sur `q` pour quitter le programme

Comme le CNN sera entraîné sur toute l'image, nous avons choisi des gestes très distincts afin de limiter les confusions entre les classes :

- `LEFT` : paume de la main gauche ouverte, avec les doigts collés ;
- `RIGHT` : paume de la main droite ouverte, avec les doigts écartés ;
- `FIRE` : poing de la main droite fermé ;
- `ENTER` : deux paumes ouvertes ;
- `QUIT` : croix avec les bras.

Nous avons ajouté une classe `NONE` où il n'y a pas de gestes à reconnaître, pour éviter que le CNN classe des images même lorsque l'utilisateur ne fait rien.

2 Bases

2.1 Neurone artificiel

Mathématiquement, un neurone est une fonction

$$f_\omega : \mathbb{R}^n \to \mathbb{R}, \qquad x \mapsto \sigma(Wx + b)$$

Le vecteur d'entrée $x = (x_1, \ldots, x_n)$ représente les caractéristiques du problème. Chaque entrée est associée à un poids w_i qui mesure l'importance de cette variable dans la décision du neurone.

La première étape consiste à calculer une combinaison linéaire des entrées :

$$z = Wx + b = \sum_{i=1}^{n} w_i x_i + b$$

Le terme b est appelé *biais* et permet de décaler la fonction de décision.

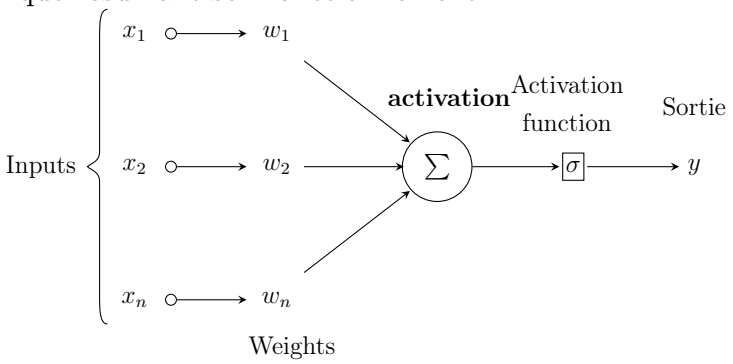
Ensuite, cette quantité z est transformée par une fonction d'activation σ. Cette fonction introduit une non-linéarité dans le modèle, ce qui permet aux réseaux de neurones d'approximer des relations complexes entre les variables.

Finalement, la sortie du neurone est donnée par

$$y = \sigma(z)$$

Ce mécanisme simple constitue l'unité de base des réseaux de neurones profonds, où plusieurs neurones sont empilés en couches afin d'apprendre des représentations de plus en plus abstraites des données.

Voici un graphique résumant son fonctionnement :



2.2 Perceptron

Un perceptron [?] est un réseau de neurones composé de deux couches :

1. une couche cachée dont tous les neurones partagent la même fonction d'activation non linéaire, généralement une sigmoïde,
2. une couche de sortie linéaire.

Le modèle est caractérisé par plusieurs paramètres structurels :

— n : le nombre de variables d'entrée,

— k : le nombre de neurones dans la couche cachée,

— p : le nombre de neurones dans la couche de sortie.

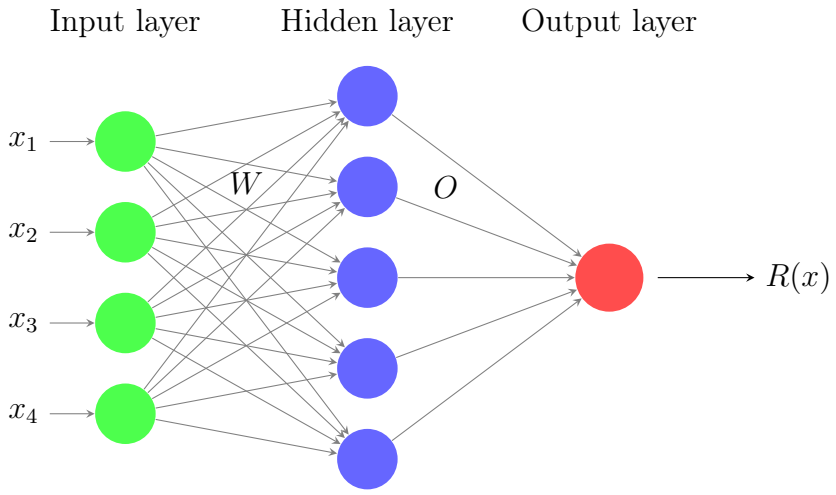


FIGURE 1 – Perceptron with $n = 4$, $k = 5$ and $p = 1$

Les paramètres du réseau sont regroupés dans deux matrices :

— $W \in \mathcal{M}_{k,n}$ qui contient les poids reliant l'entrée à la couche cachée,

— $O \in \mathcal{M}_{p,k}$ qui contient les poids reliant la couche cachée à la couche de sortie.

Pour un vecteur d'entrée $x \in \mathbb{R}^n$, le réseau applique la transformation suivante :

$$R(x) = O\, s(Wx)$$

où $s : \mathbb{R} \to \mathbb{R}$ est une fonction sigmoïde appliquée composante par composante.
Le calcul se déroule donc en deux étapes :

1. une transformation linéaire Wx suivie de l'application de la fonction d'activation s, produisant les activations de la couche cachée,
2. une combinaison linéaire de ces activations via la matrice O pour produire la sortie finale.

Les matrices W et O constituent les paramètres du réseau, appelés *poids*, qui seront appris à partir des données lors de la phase d'entraînement.

```python
import numpy as np
import matplotlib.pyplot as plt

def gradient_descent(f, grad, x0, alpha, n_iter=100):
    x = np.array(x0, dtype=float)
    traj = [x.copy()]
    for i in range(n_iter):
        x = x - alpha *grad(x)
        traj.append(x.copy())
    return np.array(traj)
```

Listing 1 – Algorithme de descente de gradient